

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG



BÁO CÁO BÀI TẬP LỚN CÁC HỆ THỐNG PHÂN TÁN

ĐỀ TÀI: PHÁT TRIỂN ỨNG DỤNG MICROSERVICE

***Sinh viên thực hiện:* Đỗ Minh Đức - B22DCCN223**

Vũ Thế Vinh - B22DCCN907

Đinh Quốc Việt - B22DCCN895

***Nhóm lớp:* 02**

***Nhóm bài tập:* 18**

***Giảng viên :* TS. Kim Ngọc Bách**

HÀ NỘI - NĂM 2026

LỜI MỞ ĐẦU

Trước tiên, chúng em xin gửi lời cảm ơn chân thành đến thầy Kim Ngọc Bách - giảng viên bộ môn, đã dành thời gian xem xét và đánh giá bài báo cáo của nhóm. Sự hướng dẫn và những góp ý quý báu của thầy đã giúp chúng em hoàn thiện bài báo cáo này một cách tốt nhất.

Chúng em cũng xin gửi lời cảm ơn đến các thành viên trong nhóm, những người đã cùng nhau hợp tác, đóng góp ý tưởng và nỗ lực hết mình để hoàn thành bài tập. Mỗi người trong nhóm đều có những đóng góp quan trọng, và sự đoàn kết, phối hợp chặt chẽ giữa các thành viên là yếu tố quyết định giúp chúng em hoàn thành công việc này.

Cảm ơn các bạn trong nhóm đã luôn hỗ trợ, chia sẻ và cùng nhau vượt qua mọi thử thách trong suốt quá trình làm việc. Chúng em hy vọng rằng bài báo cáo này sẽ đạt được kết quả tốt, phản ánh công sức mà nhóm đã bỏ ra.

Xin chân thành cảm ơn!

MỤC LỤC

DANH SÁCH BẢNG BIỂU.....	1
DANH SÁCH HÌNH ẢNH.....	2
BẢNG THUẬT NGỮ VIẾT TẮT.....	3
CHƯƠNG 1. MÔ TẢ BÀI TOÁN.....	6
1.1. Giới thiệu bài toán.....	6
1.2. Yêu cầu chức năng.....	7
1.2.1. Phía người dùng (User).....	7
1.2.2. Phía quản trị viên (Admin).....	9
1.3. Yêu cầu phi chức năng.....	10
1.4. Phạm vi hệ thống.....	12
1.4.1. Phạm vi bao gồm.....	12
1.4.2. Phạm vi không bao gồm.....	13
1.5. Đối tượng sử dụng và môi trường triển khai.....	14
1.5.1. Đối tượng sử dụng.....	14
1.5.2. Môi trường triển khai.....	15
1.6. Tổng kết chương.....	15
CHƯƠNG 2. KIẾN TRÚC HỆ THỐNG.....	17
2.1. Tổng quan kiến trúc Microservices.....	17
2.2. Sơ đồ khối tổng thể.....	17
2.3. Sơ đồ thành phần.....	18
2.4. Mô hình giao tiếp.....	19
2.5. Công nghệ sử dụng.....	20
CHƯƠNG 3. MÔ TẢ CÁC MICROSERVICE.....	21
3.1. API Gateway.....	21
3.2. User Service.....	21
3.3. Product Service.....	22
3.4. Order Service.....	22
3.5. Inventory Service.....	23
3.6. Payment Service.....	23
3.7. Notification Service.....	24
CHƯƠNG 4. THIẾT KẾ RESTFUL API.....	25
4.1. Nguyên tắc thiết kế API.....	25
4.2. Xác thực và phân quyền.....	25
4.3. Các endpoint chính.....	25
4.3.1. User Service — tiền tố /api/users.....	25
4.3.2. Product Service — tiền tố /api/products.....	26
4.3.3. Order Service — tiền tố /api/orders.....	28

4.4. Chuẩn định dạng phản hồi.....	29
CHƯƠNG 5. LUỒNG MESSAGE QUEUE.....	30
5.1. Tổng quan RabbitMQ.....	30
5.2. Topology exchange và queue.....	30
5.3. Luồng giao tiếp đồng bộ (REST).....	31
5.3.1. Sequence Diagram luồng login.....	31
5.4. Luồng giao tiếp bất đồng bộ.....	32
5.4.1. Sequence Diagram luồng Đặt hàng.....	33
5.5. Topology RabbitMQ (chi tiết kỹ thuật).....	33
5.6. Xử lý lỗi và Dead Letter Queue.....	34
CHƯƠNG 6. THỬ NGHIỆM VÀ ĐÁNH GIÁ.....	35
6.1. Môi trường thử nghiệm.....	35
6.2. Kịch bản kiểm thử.....	35
6.2.1. Kiểm thử luồng người dùng đầu cuối.....	35
6.2.2. Kiểm thử tính chịu lỗi của hệ thống phân tán.....	36
6.3. Kết quả kiểm thử.....	37
6.4. Đánh giá hệ thống.....	38
6.4.1. Ưu điểm đạt được.....	38
6.4.2. Hạn chế và hướng phát triển.....	38
CHƯƠNG 7: HƯỚNG DẪN CÀI ĐẶT.....	40
7.1. Yêu cầu môi trường.....	40
7.2. Cài đặt và khởi động hệ thống.....	41
Bước 1. Tải mã nguồn dự án.....	41
Bước 2. Khởi động hệ thống bằng Docker Compose.....	41
Bước 3. Kiểm tra trạng thái container.....	42
7.3. Truy cập hệ thống.....	42
7.4. Hướng dẫn sử dụng cơ bản.....	42
7.5. Dừng hệ thống.....	43
7.6. Một số lỗi thường gặp.....	43
KẾT LUẬN.....	45
TÀI LIỆU THAM KHẢO.....	46

DANH SÁCH BẢNG BIỂU

Bảng 1.1. Yêu cầu phi chức năng của hệ thống.....	11
Bảng 2.3. Danh sách thành phần và công nghệ trong hệ thống.....	18
Bảng 2.2. Hai mô hình giao tiếp trong hệ thống.....	19
Bảng 2.3. Công nghệ sử dụng trong dự án.....	20
Bảng 3.1. Các module chính của User Service.....	21
Bảng 4.1. Endpoint của User Service.....	25
Bảng 4.2. Endpoint của Product Service.....	26
Bảng 4.3. Endpoint của Order Service.....	28
Bảng 4.4. Cấu trúc chuẩn của API response.....	29
Bảng 5.1. Topology RabbitMQ của hệ thống.....	30
Bảng 6.1. Môi trường thử nghiệm.....	35
Bảng 6.2. Kịch bản kiểm thử luồng người dùng.....	35
Bảng 6.3. Kịch bản kiểm thử tính chịu lỗi.....	36
Bảng 7.1. Yêu cầu môi trường triển khai.....	40
Bảng 7.3. Địa chỉ truy cập hệ thống.....	42

DANH SÁCH HÌNH ẢNH

Hình 2.2. Sơ đồ kiến trúc tổng thể.....	18
Hình 5.1. Sequence Diagram luồng login.....	31
Hình 5.2. Sequence Diagram luồng xem đơn hàng.....	32
Hình 5.3. Sequence Diagram luồng giao tiếp bất đồng bộ.....	33

BẢNG THUẬT NGỮ VIẾT TẮT

Thuật ngữ viết tắt	Nghĩa tiếng Anh	Nghĩa tiếng Việt
API	Application Programming Interface	Giao diện lập trình ứng dụng, dùng để các dịch vụ kết nối và trao đổi dữ liệu với nhau.
CRUD	Create, Read, Update, Delete	Bốn thao tác cơ bản đối với dữ liệu: Tạo mới, Đọc, Cập nhật và Xóa.
DB	Database	Cơ sở dữ liệu.
DLQ	Dead Letter Queue	Hàng đợi chứa các thông điệp lỗi hoặc xử lý thất bại sau số lần thử lại tối đa.
DLX	Dead Letter Exchange	Trạm định tuyến các thông điệp lỗi vào hàng đợi Dead Letter Queue tương ứng.
DTO	Data Transfer Object	Đối tượng chuyển đổi dữ liệu giữa các tầng trong một dịch vụ.
E2E	End-to-End	Kiểm thử toàn trình, giả lập toàn bộ hành trình trải nghiệm thực tế của người dùng.
HTTP	Hypertext Transfer Protocol	Giao thức truyền tải siêu văn bản dùng cho giao tiếp đồng bộ REST.

HTTPS	Hypertext Transfer Protocol Secure	Giao thức mã hóa bảo mật dùng để gọi các dịch vụ bên ngoài an toàn như Claude API.
JPQL	Java Persistence Query Language	Ngôn ngữ truy vấn hướng đối tượng của Java, được dùng để tìm kiếm sản phẩm thông qua từ khóa (LIKE query).
JSON	JavaScript Object Notation	Định dạng dữ liệu dạng văn bản nhẹ, dùng để trao đổi thông tin giữa Client và Server.
JWT	JSON Web Token	Chuỗi mã hóa dạng Token dùng để xác thực và phân quyền người dùng tập trung tại hệ thống.
MĐ	Mã định danh	Định danh duy nhất (Identifier / ID) của người dùng hoặc thực thể trong hệ thống.
MQ	Message Queue	Hàng đợi thông điệp, đóng vai trò làm bộ đệm trung gian xử lý luồng bất đồng bộ.
REST	REpresentational State Transfer	Kiến trúc thiết kế API đồng bộ dựa trên các phương thức chuẩn của HTTP.
SEO	Search Engine Optimization	Tối ưu hóa công cụ tìm kiếm, áp dụng cho đường dẫn sản phẩm thân thiện (slug URL).
TMĐT	Thương mại điện tử	Lĩnh vực nghiệp vụ cốt lõi của bài toán hệ thống sàn mua sắm trực tuyến.

URL	Uniform Resource Locator	Đường dẫn định vị tài nguyên trên mạng internet.
WSL2	Windows Subsystem for Linux 2	Kiến trúc tích hợp nhân Linux trên Windows dùng để chạy môi trường Docker Desktop hiệu năng cao.

CHƯƠNG 1. MÔ TẢ BÀI TOÁN

1.1. Giới thiệu bài toán

Trong bối cảnh nền kinh tế số phát triển mạnh mẽ, thương mại điện tử (e-commerce) đã trở thành một trong những lĩnh vực có tốc độ tăng trưởng nhanh nhất trên toàn cầu. Theo các báo cáo thị trường, doanh số thương mại điện tử toàn cầu đã vượt ngưỡng hàng nghìn tỷ USD và tiếp tục duy trì đà tăng trưởng ổn định qua từng năm. Tại Việt Nam, thị trường thương mại điện tử cũng không nằm ngoài xu hướng đó, với sự bùng nổ của các sàn giao dịch trực tuyến lớn và hành vi mua sắm trực tuyến ngày càng phổ biến trong cộng đồng người dùng internet.

Sự tăng trưởng vũ bão của lĩnh vực thương mại điện tử đặt ra những thách thức kỹ thuật vô cùng lớn cho các nhà phát triển phần mềm. Hệ thống phải đáp ứng hàng triệu người dùng đồng thời, xử lý hàng chục nghìn giao dịch mỗi giây vào các thời điểm cao điểm như ngày lễ hay flash sale, trong khi vẫn phải đảm bảo độ trễ thấp, tính sẵn sàng cao và không để xảy ra sự cố dữ liệu. Đây là những yêu cầu mà các kiến trúc phần mềm truyền thống gặp nhiều khó khăn trong việc đáp ứng.

Kiến trúc nguyên khối (Monolithic Architecture) — mô hình phổ biến trong giai đoạn đầu của phát triển phần mềm — đặc trưng bởi việc toàn bộ logic nghiệp vụ, giao tiếp dữ liệu và giao diện người dùng được đóng gói trong một ứng dụng duy nhất và triển khai như một đơn vị. Mô hình này có ưu điểm đơn giản, dễ phát triển và kiểm thử trong giai đoạn đầu, nhưng bộc lộ nhiều nhược điểm nghiêm trọng khi hệ thống tăng trưởng về quy mô và độ phức tạp:

- Không thể mở rộng theo chiều ngang (horizontal scaling) cho từng thành phần riêng lẻ — khi module đặt hàng chịu tải cao, hệ thống buộc phải scale toàn bộ ứng dụng, gây lãng phí tài nguyên cho các module ít tải.
- Một lỗi nhỏ trong bất kỳ module nào cũng có thể gây sập toàn bộ ứng dụng, ảnh hưởng nghiêm trọng đến trải nghiệm người dùng và doanh thu doanh nghiệp.

- Chu kỳ phát triển và triển khai kéo dài do mỗi thay đổi dù nhỏ đều yêu cầu build và deploy lại toàn bộ ứng dụng, làm chậm tốc độ ra mắt tính năng mới.
- Khó áp dụng công nghệ mới cho từng phần của hệ thống do toàn bộ ứng dụng sử dụng chung một ngôn ngữ và framework.
- Codebase ngày càng phình to, khó hiểu và khó bảo trì khi đội ngũ phát triển và số lượng tính năng tăng lên theo thời gian.

Trước những hạn chế này, kiến trúc Microservices đã nổi lên như một giải pháp kiến trúc hiện đại, được các công ty công nghệ hàng đầu như Netflix, Amazon, Uber, Spotify áp dụng thành công để xây dựng những hệ thống có khả năng phục vụ hàng trăm triệu người dùng trên toàn cầu. Kiến trúc Microservices phân rã ứng dụng thành các dịch vụ nhỏ, độc lập, mỗi dịch vụ đảm nhiệm một miền nghiệp vụ cụ thể, có thể phát triển, triển khai và mở rộng hoàn toàn độc lập với nhau.

Đồ án này được thực hiện nhằm xây dựng một sàn thương mại điện tử hoàn chỉnh theo kiến trúc Microservices, áp dụng các nguyên lý của hệ thống phân tán vào bài toán thực tế. Hệ thống cung cấp đầy đủ chức năng cho cả người dùng mua sắm lẫn quản trị viên, đồng thời minh họa cách xử lý những thách thức kỹ thuật phổ biến trong môi trường phân tán như đảm bảo tính nhất quán dữ liệu, giao tiếp bất đồng bộ, xác thực tập trung và chịu lỗi độc lập giữa các service.

1.2. Yêu cầu chức năng

Hệ thống được thiết kế để phục vụ hai nhóm người dùng chính với các nhu cầu và quyền hạn khác nhau: người dùng cuối (khách hàng mua sắm) và quản trị viên (admin). Mỗi nhóm có tập chức năng riêng, được phân quyền rõ ràng thông qua cơ chế JWT và role-based access control.

1.2.1. Phía người dùng (User)

Người dùng cuối là đối tượng trực tiếp tương tác với giao diện mua sắm. Toàn bộ các chức năng dành cho người dùng được thiết kế theo hướng trực quan, tiện lợi và đảm bảo trải nghiệm liền mạch xuyên suốt hành trình mua sắm từ khi tìm kiếm sản phẩm đến khi hoàn tất đơn hàng.

a) Quản lý tài khoản:

- Đăng ký tài khoản mới bằng email và mật khẩu; mật khẩu được mã hóa bằng BCrypt trước khi lưu vào cơ sở dữ liệu.
- Đăng nhập và nhận JSON Web Token (JWT) — token được sử dụng để xác thực mọi request tiếp theo.
- Xem và cập nhật thông tin hồ sơ cá nhân bao gồm tên hiển thị và số điện thoại.
- Đổi mật khẩu với yêu cầu xác nhận mật khẩu cũ để đảm bảo bảo mật.
- Quản lý sở địa chỉ giao hàng: thêm, xem và xóa địa chỉ, hỗ trợ tối đa 5 địa chỉ mỗi tài khoản.

b) Duyệt và tìm kiếm sản phẩm:

- Xem danh sách sản phẩm với phân trang, hỗ trợ lọc theo danh mục, thương hiệu và khoảng giá.
- Tìm kiếm sản phẩm theo từ khóa với kết quả được lọc và sắp xếp theo độ liên quan.
- Xem chi tiết sản phẩm theo đường dẫn slug URL thân thiện (SEO-friendly).
- Xem đánh giá và bình luận của người mua trước về sản phẩm.

c) Quản lý giỏ hàng:

- Thêm sản phẩm vào giỏ hàng với số lượng tùy chọn.
- Cập nhật số lượng hoặc xóa sản phẩm khỏi giỏ hàng.
- Giỏ hàng được lưu bền vững trong cơ sở dữ liệu — không bị mất khi người dùng đóng trình duyệt hay reload trang.

d) Đặt hàng và thanh toán:

- Tạo đơn hàng từ giỏ hàng hiện tại, hệ thống tự động kiểm tra tồn kho trước khi xác nhận.
- Áp dụng mã giảm giá (coupon) trước khi đặt hàng; hỗ trợ hai loại: giảm giá cố định (fixed) và giảm theo phần trăm (percentage).
- Xem trước giá trị giảm giá của coupon trước khi áp dụng vào đơn hàng.
- Xem lịch sử đơn hàng và theo dõi trạng thái đơn hàng theo thời gian thực.
- Xem chi tiết từng đơn hàng bao gồm sản phẩm, số lượng, giá và trạng thái xử lý.

e) Đánh giá sản phẩm:

- Viết đánh giá sản phẩm sau khi mua hàng thành công với thang điểm 1–5 sao và bình luận văn bản.
- Chỉnh sửa đánh giá trong cửa sổ 24 giờ kể từ thời điểm đăng.
- Xem toàn bộ đánh giá của sản phẩm từ cộng đồng người mua.

f) Chatbot AI hỗ trợ tư vấn:

- Tương tác với chatbot trí tuệ nhân tạo được tích hợp Claude API để nhận tư vấn sản phẩm theo nhu cầu.
- Chatbot phản hồi theo thời gian thực (streaming response), được tích hợp ngữ cảnh về catalog sản phẩm để tư vấn chính xác và phù hợp.

1.2.2. Phía quản trị viên (Admin)

Quản trị viên có quyền truy cập vào bảng điều khiển quản lý toàn diện, cho phép giám sát và điều hành mọi hoạt động của hệ thống. Giao diện admin được tách

biệt hoàn toàn với giao diện người dùng thông thường, với các endpoint được bảo vệ bởi phân quyền ADMIN.

- Quản lý danh mục và sản phẩm: thực hiện đầy đủ thao tác CRUD (tạo, đọc, cập nhật, xóa) cho danh mục sản phẩm và thông tin sản phẩm.
- Quản lý tồn kho: xem và cập nhật số lượng tồn kho theo từng sản phẩm; nhận cảnh báo sản phẩm sắp hết hàng dưới ngưỡng cấu hình sẵn.
- Quản lý đơn hàng: xem chi tiết đơn hàng, cập nhật trạng thái (pending → processing → shipped → delivered), lọc đơn hàng theo khoảng thời gian.
- Quản lý người dùng: xem danh sách tài khoản, cập nhật thông tin và phân quyền người dùng trong hệ thống.
- Quản lý mã giảm giá: tạo mã giảm giá mới, kích hoạt hoặc hủy kích hoạt mã, theo dõi lịch sử sử dụng và mức độ hiệu quả.
- Kiểm duyệt đánh giá: xem toàn bộ đánh giá sản phẩm từ người dùng, ẩn hoặc hiển thị đánh giá theo chính sách nội dung.
- Dashboard thống kê: xem biểu đồ doanh thu, số đơn hàng, sản phẩm bán chạy và lượng người dùng mới đăng ký theo khoảng thời gian tùy chọn.
- Quản lý thông báo: xem log lịch sử thông báo đã gửi tới người dùng, lọc theo trạng thái và khoảng thời gian.
- Xem lịch sử giao dịch thanh toán: xem tất cả giao dịch, lọc theo trạng thái (pending, completed, failed) và thời gian.

1.3. Yêu cầu phi chức năng

Bên cạnh các yêu cầu chức năng, hệ thống cần đáp ứng một tập hợp các yêu cầu phi chức năng quan trọng — những yêu cầu này không phản ánh trực tiếp nghiệp vụ mà phản ánh chất lượng vận hành và khả năng phát triển bền vững của hệ thống. Với bối cảnh thương mại điện tử đòi hỏi độ tin cậy cao, các yêu cầu phi chức năng dưới đây được xác định là những thuộc tính chất lượng cốt lõi của hệ thống.

Bảng 1.1. Yêu cầu phi chức năng của hệ thống

Thuộc tính	Yêu cầu cụ thể	Giải pháp kỹ thuật áp dụng
Khả năng mở rộng	Mỗi microservice có thể scale độc lập theo chiều ngang mà không ảnh hưởng đến service khác	Database-per-service; Docker Compose scale; kiến trúc stateless
Tính sẵn sàng cao	Một service gặp sự cố không làm sập toàn bộ hệ thống; không mất dữ liệu khi service tạm ngừng	Message queue RabbitMQ với durable queue; Dead Letter Queue; idempotency consumer
Bảo mật	Xác thực tập trung; chống giả mạo định danh; phân quyền rõ ràng theo role	JWT HS256 verify tại API Gateway; overwrite X-User-Id; role guard tại từng service
Hiệu năng	API phản hồi nhanh; luồng bất đồng bộ không chặn client chờ đợi	Spring WebFlux reactive tại Gateway; publish-subscribe async qua RabbitMQ
Nhất quán dữ liệu	Không truy vấn chéo database; đồng bộ qua event và REST nội bộ	Database-per-service với PostgreSQL riêng; event-driven synchronization

Khả năng quan sát	Theo dõi luồng request xuyên service; giám sát queue và message	Trace ID xuyên suốt request; RabbitMQ Management UI tại port 15672
Dễ triển khai	Toàn bộ hệ thống khởi chạy bằng một lệnh; schema tự động migrate	Docker Compose; Flyway versioned migration (V1__, V2__...)

Khả năng mở rộng là yêu cầu nền tảng nhất trong bối cảnh thương mại điện tử, nơi lưu lượng truy cập có thể tăng đột biến gấp nhiều lần vào các sự kiện mua sắm đặc biệt. Kiến trúc Microservices cho phép chỉ scale riêng những service đang chịu tải cao (ví dụ: order-service và payment-service trong flash sale) thay vì phải scale toàn bộ hệ thống, giúp tối ưu hóa chi phí hạ tầng đáng kể.

Tính sẵn sàng cao đặc biệt quan trọng với các hệ thống giao dịch trực tuyến — thời gian chết (downtime) đồng nghĩa với mất doanh thu và mất niềm tin của khách hàng. Hệ thống được thiết kế để một service gặp sự cố không gây hiệu ứng domino kéo sập toàn bộ, và message queue đảm bảo không có dữ liệu nào bị mất trong quá trình sự cố xảy ra.

1.4. Phạm vi hệ thống

1.4.1. Phạm vi bao gồm

Hệ thống được xây dựng bao gồm đầy đủ các thành phần sau đây, tạo thành một hệ sinh thái hoàn chỉnh có khả năng vận hành độc lập:

Bảng 1.2. Tổng quan các thành phần trong phạm vi hệ thống

Thành phần	Số lượng	Công nghệ	Mô tả
------------	----------	-----------	-------

Backend microservice	6 service	Java 17 / Spring Boot 3	user, product, order, payment, inventory, notification
API Gateway	1	Spring Cloud Gateway (WebFlux)	Điểm vào duy nhất, xác thực JWT tại biên
Frontend	1	Next.js 14 / TypeScript	Giao diện người dùng và admin
Cơ sở dữ liệu	7 PostgreSQL	PostgreSQL 15 + Flyway	Mỗi service một database độc lập
Message Broker	1	RabbitMQ 3	Giao tiếp bất đồng bộ giữa các service
Container hóa	15 container	Docker + Docker Compose	Toàn bộ hệ thống đóng gói và khởi chạy

Về mặt chức năng, hệ thống bao gồm toàn bộ hành trình mua sắm từ đăng ký tài khoản, duyệt sản phẩm, quản lý giỏ hàng, đặt hàng, thanh toán, theo dõi đơn hàng đến đánh giá sản phẩm. Phía quản trị bao gồm đầy đủ công cụ quản lý catalog, tồn kho, đơn hàng, người dùng, mã giảm giá và dashboard thống kê trực quan. Tính năng chatbot AI tích hợp Claude API là điểm nổi bật, mang lại trải nghiệm tư vấn sản phẩm thông minh cho người dùng.

1.4.2. Phạm vi không bao gồm

Để giữ phạm vi đề án ở mức phù hợp với thời gian và nguồn lực thực hiện, một số tính năng và hạ tầng sau đây không nằm trong phạm vi triển khai của đề án này:

- Tích hợp cổng thanh toán thực (VNPay, Stripe, MoMo): hệ thống sử dụng mock payment để mô phỏng luồng thanh toán. Cấu trúc dữ liệu PaymentSession và PaymentTransaction đã được thiết kế sẵn sàng để tích hợp thực tế trong tương lai.

- Triển khai lên môi trường cloud production: hệ thống được thiết kế và kiểm thử hoàn toàn trên môi trường local bằng Docker Compose. Việc triển khai lên Kubernetes trên các cloud provider (AWS, GCP, Azure) là hướng phát triển tiếp theo.
- Hệ thống giao hàng thực tế: đồ án không tích hợp với bất kỳ đơn vị vận chuyển nào; trạng thái đơn hàng được cập nhật thủ công bởi admin.
- Xử lý dữ liệu thời gian thực (streaming): hệ thống không sử dụng Apache Kafka hay Apache Flink cho việc xử lý luồng dữ liệu lớn theo thời gian thực.
- Phân tán dữ liệu địa lý (geo-distribution): không triển khai multi-region hay CDN cho static assets.
- Tìm kiếm nâng cao: không tích hợp Elasticsearch; tìm kiếm sản phẩm sử dụng JPQL LIKE query trên PostgreSQL.

1.5. Đối tượng sử dụng và môi trường triển khai

1.5.1. Đối tượng sử dụng

Hệ thống được xây dựng hướng tới ba nhóm đối tượng sử dụng chính với nhu cầu và mức độ tương tác khác nhau:

- Khách hàng (Customer): Người dùng cuối tương tác với giao diện mua sắm thông qua trình duyệt web hoặc thiết bị di động. Nhóm này không yêu cầu kiến thức kỹ thuật, chỉ cần tài khoản đã đăng ký để trải nghiệm đầy đủ tính năng mua sắm.
- Quản trị viên (Administrator): Nhân viên vận hành hệ thống với quyền truy cập đặc biệt vào bảng điều khiển quản lý. Nhóm này cần được cấp quyền ADMIN trong hệ thống và có kiến thức cơ bản về quản lý thương mại điện tử.
- Nhà phát triển (Developer): Nhóm kỹ thuật xây dựng, bảo trì và mở rộng hệ thống. Đây là đối tượng phục vụ chính của đồ án — những sinh viên và kỹ

sư phần mềm muốn học hỏi về kiến trúc Microservices và hệ thống phân tán thông qua một ví dụ thực tế hoàn chỉnh.

1.5.2. Môi trường triển khai

Một trong những mục tiêu thiết kế quan trọng của đề án là đảm bảo hệ thống dễ dàng khởi chạy trên bất kỳ máy tính phát triển nào mà không yêu cầu cấu hình phức tạp. Toàn bộ hệ thống được đóng gói trong Docker và khởi chạy bằng một lệnh duy nhất:

`docker-compose up --build`

Lệnh này sẽ khởi động toàn bộ 15 container bao gồm 7 PostgreSQL database, 1 RabbitMQ broker, 6 backend microservice và 1 frontend. Flyway tự động chạy migration để tạo schema và seed dữ liệu mẫu ngay khi mỗi service khởi động, đảm bảo môi trường phát triển luôn nhất quán giữa các thành viên trong nhóm.

Bảng 1.3. Các cổng truy cập công khai của hệ thống

Cổng truy cập	Thành phần	Mục đích
:3000	Frontend (Next.js)	Giao diện người dùng và admin
:8080	API Gateway	Cổng vào duy nhất cho toàn bộ REST API
:15672	RabbitMQ Management UI	Theo dõi queue, exchange và message
Nội bộ	Các backend service	Không expose trực tiếp ra ngoài Docker network

1.6. Tổng kết chương

Chương 1 đã trình bày toàn diện bài toán được đặt ra trong đề án này. Xuất phát từ thực tế phát triển mạnh mẽ của thương mại điện tử và những hạn chế của kiến trúc nguyên khối truyền thống, đề án đề xuất xây dựng một sàn thương mại điện tử theo kiến trúc Microservices như một giải pháp hiện đại, có khả năng mở rộng và chịu lỗi tốt hơn.

Hệ thống được xác định với tập yêu cầu chức năng đầy đủ cho cả người dùng cuối (quản lý tài khoản, mua sắm, đặt hàng, đánh giá, chatbot AI) và quản trị viên (quản lý catalog, tồn kho, đơn hàng, người dùng, thống kê). Các yêu cầu phi chức năng cốt lõi bao gồm khả năng mở rộng độc lập, tính sẵn sàng cao, bảo mật chặt chẽ và dễ triển khai được xác định rõ ràng cùng với giải pháp kỹ thuật tương ứng.

Phạm vi hệ thống được giới hạn phù hợp với mục tiêu học thuật của đề án: bao gồm đầy đủ các thành phần kỹ thuật cần thiết để minh họa kiến trúc Microservices và hệ thống phân tán, đồng thời loại trừ các tích hợp thực tế phức tạp (cổng thanh toán, cloud deployment) để tập trung vào việc thể hiện các nguyên lý kiến trúc một cách rõ ràng và hoàn chỉnh nhất. Các chương tiếp theo sẽ đi sâu vào thiết kế kiến trúc, triển khai từng microservice và đánh giá kết quả thực nghiệm.

CHƯƠNG 2. KIẾN TRÚC HỆ THỐNG

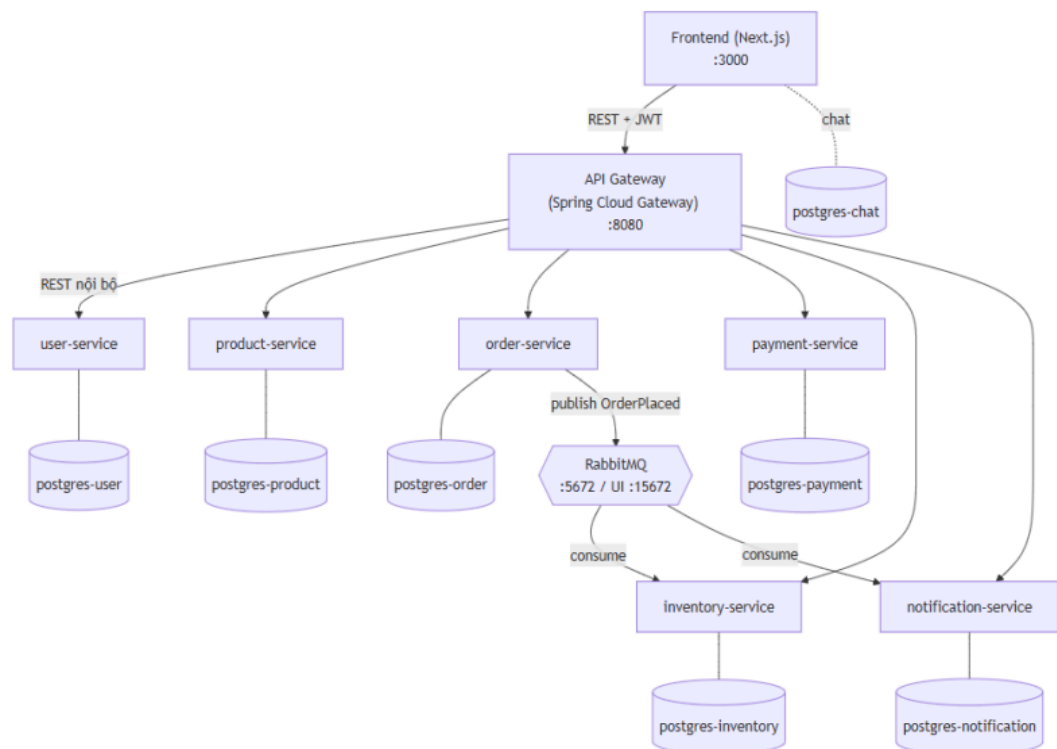
2.1. Tổng quan kiến trúc Microservices

Kiến trúc Microservices phân rã ứng dụng thành các dịch vụ nhỏ, mỗi dịch vụ độc lập về triển khai, vòng đời phát triển và lưu trữ dữ liệu. Mỗi service chịu trách nhiệm một miền nghiệp vụ rõ ràng (Single Responsibility Principle) và giao tiếp với nhau qua giao thức nhẹ (REST/HTTP hoặc message queue).

Hệ thống PTIT HTPT tuân thủ ba nguyên tắc cốt lõi của microservices: (1) API Gateway là điểm vào duy nhất — không service nào expose port ra ngoài Docker network mà bỏ qua Gateway; (2) Database-per-service — mỗi service có PostgreSQL riêng biệt, không service nào truy vấn trực tiếp database của service khác; (3) Loose coupling qua message queue — giao tiếp bất đồng bộ qua RabbitMQ cho các nghiệp vụ không cần kết quả tức thì.

2.2. Sơ đồ khối tổng thể

Để đáp ứng các yêu cầu phi chức năng về tính sẵn sàng cao, khả năng mở rộng độc lập và cô lập dữ liệu, hệ thống được thiết kế theo kiến trúc Microservices. Toàn bộ các thành phần backend được đóng gói dưới dạng các Docker Container riêng biệt và giao tiếp qua hai giao thức: REST đồng bộ và Message Queue bất đồng bộ.



Hình 2.2. Sơ đồ kiến trúc tổng thể

2.3. Sơ đồ thành phần

Hệ thống bao gồm các thành phần được liệt kê trong Bảng 2.1. Các service backend không expose port ra host — chỉ có Frontend (:3000), API Gateway (:8080) và RabbitMQ Management UI (:15672) được truy cập từ trình duyệt.

Bảng 2.3. Danh sách thành phần và công nghệ trong hệ thống

Thành phần	Công nghệ	Port	Vai trò chính
Frontend	Next.js 14 / TypeScript	3000	Giao diện người dùng và admin
API Gateway	Spring Cloud Gateway (WebFlux)	8080	Cổng vào duy nhất, xác thực JWT tại biên

user-service	Spring Boot 3 / Java 17	Nội bộ	Đăng ký, đăng nhập, hồ sơ, địa chỉ, phát hành JWT
product-service	Spring Boot 3 / Java 17	Nội bộ	Danh mục, sản phẩm, tìm kiếm, đánh giá
order-service	Spring Boot 3 / Java 17	Nội bộ	Giỏ hàng, đặt hàng, coupon, publish event
payment-service	Spring Boot 3 / Java 17	Nội bộ	Phiên và giao dịch thanh toán
inventory-service	Spring Boot 3 / Java 17	Nội bộ	Quản lý tồn kho, consume event OrderPlaced
notification-service	Spring Boot 3 / Java 17	Nội bộ	Ghi log thông báo, consume event OrderPlaced
RabbitMQ	RabbitMQ 3	5672 / 15672	Message broker cho giao tiếp bất đồng bộ
PostgreSQL (x7)	PostgreSQL 15 + Flyway	Nội bộ	Mỗi service một database riêng biệt độc lập

2.4. Mô hình giao tiếp

Hệ thống sử dụng song song hai mô hình giao tiếp tùy theo đặc tính của từng nghiệp vụ, được tóm tắt trong Bảng 2.2.

Bảng 2.2. Hai mô hình giao tiếp trong hệ thống

Kiểu giao tiếp	Khi nào dùng	Ví dụ trong hệ thống
REST đồng bộ (sync)	Client cần kết quả ngay; thao tác đọc/ghi tức thời	Đăng nhập, xem sản phẩm, kiểm tra tồn kho khi đặt hàng
Message Queue bất đồng bộ (async)	Xử lý nền, không bắt client chờ; tách rời service	Sau khi đặt hàng: trừ kho + ghi log thông báo

Khi client gửi request, Frontend gọi tới API Gateway qua HTTP kèm JWT trong header Authorization. Gateway xác thực token, xóa X-User-Id do client tự gửi, chèn lại X-User-Id từ claim sub trong JWT đã verify, rồi định tuyến request tới service nội bộ tương ứng. Cơ chế này ngăn client giả mạo định danh người dùng khác.

2.5. Công nghệ sử dụng

Bảng 2.3. Công nghệ sử dụng trong dự án

Tầng	Công nghệ	Lý do lựa chọn
Backend	Java 17 + Spring Boot 3	Hệ sinh thái trưởng thành; Spring Cloud hỗ trợ microservices tốt
API Gateway	Spring Cloud Gateway (WebFlux)	Reactive, không blocking; hỗ trợ JWT filter và routing linh hoạt
Frontend	Next.js 14 + TypeScript	App Router, Server Components, type-safe, tích hợp tốt với REST API
Database	PostgreSQL 15 + Spring Data JPA	Quan hệ mạnh mẽ, hỗ trợ tốt cho nghiệp vụ thương mại điện tử
Migration	Flyway	Migration versioned (V1__, V2__...), tự động chạy khi service khởi động
Message Queue	RabbitMQ 3 (topic exchange)	Topic exchange linh hoạt; Management UI trực quan; hỗ trợ DLQ
Xác thực	JWT (HS256)	Stateless; verify tại Gateway không cần gọi thêm user-service
Container hóa	Docker + Docker Compose	Môi trường đồng nhất; khởi chạy toàn hệ thống bằng một lệnh

CHƯƠNG 3. MÔ TẢ CÁC MICROSERVICE

Mỗi microservice được tổ chức theo kiến trúc phân lớp chuẩn: web (Controller) → service (Business Logic) → repository (Data Access) → domain (Entity/DTO). Các service đều implement `TraceIdFilter` để gắn X-Trace-Id vào mọi response header, giúp theo dõi luồng request xuyên suốt nhiều service.

3.1. API Gateway

API Gateway (Spring Cloud Gateway) là thành phần biên (edge) duy nhất nhận request từ client. Service này không lưu trạng thái và không truy vấn database, đảm bảo hiệu năng cao nhờ kiến trúc reactive (WebFlux). Nó thực hiện ba nhiệm vụ chính:

- Định tuyến (Routing): ánh xạ đường dẫn `/api/users/**` → `user-service`; `/api/products/**` → `product-service`; `/api/orders/**` → `order-service`; `/api/payments/**` → `payment-service`; `/api/inventory/**` → `inventory-service`; `/api/notifications/**` → `notification-service`.
- Xác thực JWT tại biên (Edge Authentication): `JwtAuthenticationFilter` verify chữ ký token trước khi cho request đi qua. Token hết hạn hoặc sai chữ ký sẽ nhận 401 Unauthorized ngay tại Gateway mà không bao giờ chạm tới backend.
- Chống giả mạo định danh: Gateway xóa mọi header `X-User-Id` do client tự gửi lên, sau đó tự chèn lại `X-User-Id` lấy từ claim sub trong JWT đã verify. Các service nội bộ chỉ tin tưởng `X-User-Id` đến từ Gateway.

3.2. User Service

User Service quản lý toàn bộ vòng đời tài khoản người dùng. Đây là service duy nhất phát hành JWT — sau khi xác thực thành công bằng email và mật khẩu (BCrypt hashing), service tạo access token chứa `userId` (claim sub) và danh sách roles.

Bảng 3.1. Các module chính của User Service

Module	Chức năng chính
AuthService	Đăng ký tài khoản, đăng nhập, phát hành JWT

UserCrudService	CRUD thông tin người dùng dành cho admin
UserProfileService	Cập nhật hồ sơ cá nhân (tên, số điện thoại)
UserPasswordService	Đổi mật khẩu (yêu cầu xác nhận mật khẩu cũ)
AddressService	Quản lý sổ địa chỉ (tối đa 5 địa chỉ mỗi tài khoản)
UserChartsService	Thống kê số người dùng mới theo khoảng thời gian (dashboard admin)

3.3. Product Service

Product Service quản lý toàn bộ catalog sản phẩm và hệ thống đánh giá. Database product_svc chứa các bảng chính: products, categories, reviews. Các trường avg_rating và review_count trên bảng products được cập nhật tự động mỗi khi có đánh giá mới được thêm hoặc chỉnh sửa.

- ProductCrudService: CRUD sản phẩm, tìm kiếm theo từ khóa (JPQL LIKE), lọc theo danh mục và thương hiệu, phân trang theo trang.
- ReviewService: Tạo đánh giá sau khi mua hàng; cho phép chỉnh sửa trong cửa sổ 24 giờ; tự động cập nhật điểm trung bình sản phẩm.
- LowStockService: Truy vấn và trả về danh sách sản phẩm có tồn kho dưới ngưỡng cảnh báo cấu hình sẵn.
- ProductBatchService: Lấy thông tin nhiều sản phẩm trong một lần gọi qua endpoint /api/products/batch, phục vụ order-service khi hiển thị chi tiết đơn hàng.
- AdminChartsService / AdminStatsService: Tổng hợp thống kê sản phẩm bán chạy, doanh thu theo danh mục cho dashboard admin.

3.4. Order Service

Order Service là service nghiệp vụ phức tạp nhất trong hệ thống, quản lý giỏ hàng, đơn hàng và mã giảm giá. Đây cũng là service duy nhất đóng vai trò producer, publish sự kiện lên RabbitMQ sau khi đơn hàng được tạo thành công.

- CartCrudService: Quản lý giỏ hàng theo userId, lưu trữ bền vững trong database thay vì session/localStorage — giỏ hàng không mất khi người dùng đóng trình duyệt.
- OrderCrudService: Tạo đơn hàng — kiểm tra tồn kho đồng bộ với product-service, lưu đơn vào DB trong transaction, publish sự kiện OrderPlaced, trả 201 cho client mà không chờ các consumer xử lý xong.
- CouponService / CouponRedemptionService: Quản lý mã giảm giá kiểu fixed (giảm cố định) và percentage (phần trăm); kiểm soát race condition khi nhiều người dùng cùng sử dụng một coupon bằng cơ chế locking ở tầng database.
- OrderChartsService: Tổng hợp doanh thu và phân bố trạng thái đơn hàng theo khoảng thời gian tùy chọn cho biểu đồ admin.

3.5. Inventory Service

Inventory Service quản lý số lượng tồn kho theo từng sản phẩm và là consumer của sự kiện OrderPlaced từ RabbitMQ. Database inventory_svc chứa bảng inventory (số lượng hiện tại), stock_ledger (lịch sử thay đổi) và processed_events (đảm bảo idempotency).

- InventoryCrudService: CRUD số lượng tồn kho cho admin; API kiểm tra tồn kho cho internal call từ order-service.
- OrderPlacedListener: Lắng nghe queue inventory.order-events; kiểm tra eventId trong bảng processed_events trước khi xử lý (idempotency) — tránh trừ kho hai lần khi message bị retry; trừ số lượng trong bảng inventory và ghi bản ghi vào stock_ledger.

3.6. Payment Service

Payment Service quản lý phiên thanh toán (PaymentSession) và giao dịch (PaymentTransaction). Trong phạm vi đề án, thanh toán được mock — service tạo phiên thanh toán, ghi nhận giao dịch thành công mà không kết nối cổng thanh toán thực tế, nhưng toàn bộ luồng nghiệp vụ và cấu trúc dữ liệu đã sẵn sàng để tích hợp sau.

- **PaymentCrudService:** Tạo phiên thanh toán mới, xác nhận giao dịch, truy vấn lịch sử theo orderId hoặc userId.
- **AdminPaymentController:** API cho admin xem tất cả giao dịch, lọc theo trạng thái (pending, completed, failed) và khoảng thời gian.

3.7. Notification Service

Notification Service ghi nhật ký thông báo được gửi đến người dùng sau các sự kiện nghiệp vụ quan trọng. Cũng là consumer của sự kiện OrderPlaced, service này ghi bản ghi vào bảng dispatch_log với nội dung thông báo được tạo theo template định sẵn.

- **OrderPlacedNotifyListener:** Lắng nghe queue notification.order-events; kiểm tra idempotency qua bảng processed_events; tạo nội dung thông báo từ template; lưu vào dispatch_log với trạng thái dispatched.
- **NotificationCrudService / NotificationDispatchService:** CRUD thông báo và quản lý template thông báo cho các loại sự kiện khác nhau.
- **AdminNotificationController:** API admin xem log thông báo, lọc theo userId, trạng thái và khoảng thời gian.

CHƯƠNG 4. THIẾT KẾ RESTFUL API

4.1. Nguyên tắc thiết kế API

Toàn bộ API tuân thủ các nguyên tắc RESTful: sử dụng đúng HTTP method theo ngữ nghĩa (GET để đọc, POST để tạo mới, PUT/PATCH để cập nhật, DELETE để xóa); đặt tên endpoint theo danh từ số nhiều (/products, /orders, /users); trả HTTP status code đúng ngữ nghĩa; và chuẩn hóa định dạng phản hồi thống nhất toàn hệ thống.

Mọi request cần xác thực đều phải kèm header Authorization: Bearer <JWT>. API Gateway kiểm tra tính hợp lệ của token trước khi định tuyến. Với các endpoint public (đăng nhập, đăng ký, xem danh sách sản phẩm, xem đánh giá), Gateway bỏ qua bước verify JWT và chuyển tiếp trực tiếp.

4.2. Xác thực và phân quyền

Hệ thống phân quyền theo role được nhúng trong JWT payload (claim roles). JwtRoleGuard ở từng service kiểm tra danh sách role của người dùng trước khi cho phép truy cập endpoint yêu cầu quyền đặc biệt. Mô hình bảo mật hai lớp hoạt động như sau:

- Lớp 1 — API Gateway (Edge Auth): Kiểm tra JWT hợp lệ và chưa hết hạn.
Nếu token lỗi → trả 401, request không chạm backend.
- Lớp 2 — Service (Role Guard): Kiểm tra role phù hợp (ADMIN hoặc USER) với endpoint đang gọi. Nếu không đủ quyền → trả 403 Forbidden.

Cơ chế chèn X-User-Id tại Gateway và việc các service backend chỉ đọc userId từ header này (không từ request body) đảm bảo người dùng không thể thực hiện thao tác thay mặt người dùng khác.

4.3. Các endpoint chính

4.3.1. User Service — tiền tố /api/users

Bảng 4.1. Endpoint của User Service

Method	Endpoint	Mô tả	Quyền
--------	----------	-------	-------

POST	/auth/register	Đăng ký tài khoản mới	Public
POST	/auth/login	Đăng nhập, nhận JWT	Public
GET	/me	Xem thông tin bản thân	User
PUT	/me	Cập nhật hồ sơ cá nhân	User
PUT	/me/password	Đổi mật khẩu	User
GET	/me/addresses	Xem danh sách địa chỉ	User
POST	/me/addresses	Thêm địa chỉ mới	User
DELETE	/me/addresses/{id}	Xóa địa chỉ	User
GET	/admin/users	Danh sách tất cả người dùng	Admin
PATCH	/admin/users/{id}	Cập nhật thông tin người dùng	Admin

4.3.2. Product Service — tiền tố /api/products

Bảng 4.2. Endpoint của Product Service

Method	Endpoint	Mô tả	Quyền
GET	/	Danh sách sản phẩm (lọc, tìm kiếm, phân trang)	Public

GET	/ {slug}	Chi tiết sản phẩm theo slug URL	Public
POST	/batch	Lấy nhiều sản phẩm theo danh sách ID	Internal
GET	/categories	Danh sách danh mục sản phẩm	Public
POST	/admin/products	Tạo sản phẩm mới	Admin
PUT	/admin/products/ {id}	Cập nhật sản phẩm	Admin
DELETE	/admin/products/ {id}	Xóa sản phẩm	Admin
GET	/admin/low-stock	Sản phẩm sắp hết hàng	Admin
POST	/ {id}/reviews	Viết đánh giá sản phẩm	User
GET	/ {id}/reviews	Xem đánh giá sản phẩm	Public
GET	/admin/reviews	Quản lý tất cả đánh giá	Admin

4.3.3. Order Service — tiền tố `/api/orders`

Bảng 4.3. Endpoint của Order Service

Method	Endpoint	Mô tả	Quyền
GET	<code>/cart</code>	Xem giỏ hàng hiện tại	User
POST	<code>/cart/items</code>	Thêm sản phẩm vào giỏ	User
PUT	<code>/cart/items/{id}</code>	Cập nhật số lượng sản phẩm trong giỏ	User
DELETE	<code>/cart/items/{id}</code>	Xóa sản phẩm khỏi giỏ	User
POST	<code>/</code>	Tạo đơn hàng từ giỏ hàng	User
GET	<code>/</code>	Lịch sử đơn hàng của người dùng	User
GET	<code>/ {id}</code>	Chi tiết đơn hàng	User
POST	<code>/coupons/preview</code>	Xem trước giá trị giảm của coupon	User
GET	<code>/admin/orders</code>	Tất cả đơn hàng (có lọc, phân trang)	Admin
PATCH	<code>/admin/orders/{id}/status</code>	Cập nhật trạng thái đơn hàng	Admin

GET	/admin/coupons	Danh sách mã giảm giá	Admin
POST	/admin/coupons	Tạo mã giảm giá mới	Admin

4.4. Chuẩn định dạng phản hồi

Tất cả response đều được bọc trong ApiResponse wrapper nhất quán để frontend dễ xử lý:

Bảng 4.4. Cấu trúc chuẩn của API response

Trường	Kiểu	Ý nghĩa
success	boolean	true nếu thành công, false nếu có lỗi
data	object array null	Dữ liệu trả về khi thành công
message	string	Thông báo kết quả (ví dụ: "Created successfully")
errors	FieldErrorItem[]	Danh sách lỗi chi tiết theo từng trường (chỉ có khi lỗi validation)

HTTP status code luôn phản ánh đúng ngữ nghĩa: 200 OK (đọc thành công), 201 Created (tạo mới thành công), 400 Bad Request (lỗi validation đầu vào), 401 Unauthorized (chưa đăng nhập hoặc token hết hạn), 403 Forbidden (không đủ quyền), 404 Not Found (resource không tồn tại), 409 Conflict (vi phạm ràng buộc như hết hàng, coupon đã dùng).

CHƯƠNG 5. LUỒNG MESSAGE QUEUE

5.1. Tổng quan RabbitMQ

RabbitMQ là message broker trung gian đứng giữa producer (order-service) và các consumer (inventory-service, notification-service). Việc sử dụng message queue cho phép các service giao tiếp bất đồng bộ, hoàn toàn tách rời nhau: producer không cần biết consumer là ai và consumer không cần online khi producer gửi message.

Đây là thành phần then chốt giúp hệ thống đạt tính chịu lỗi độc lập: nếu notification-service tạm thời ngừng hoạt động do lỗi hoặc cập nhật, message vẫn nằm trong queue và sẽ được xử lý đầy đủ khi service khởi động lại, trong khi order-service vẫn phục vụ đặt hàng bình thường mà không bị ảnh hưởng.

5.2. Topology exchange và queue

Hệ thống sử dụng topic exchange để định tuyến message linh hoạt theo routing key. Cấu trúc đầy đủ được mô tả trong Bảng 5.1.

Bảng 5.1. Topology RabbitMQ của hệ thống

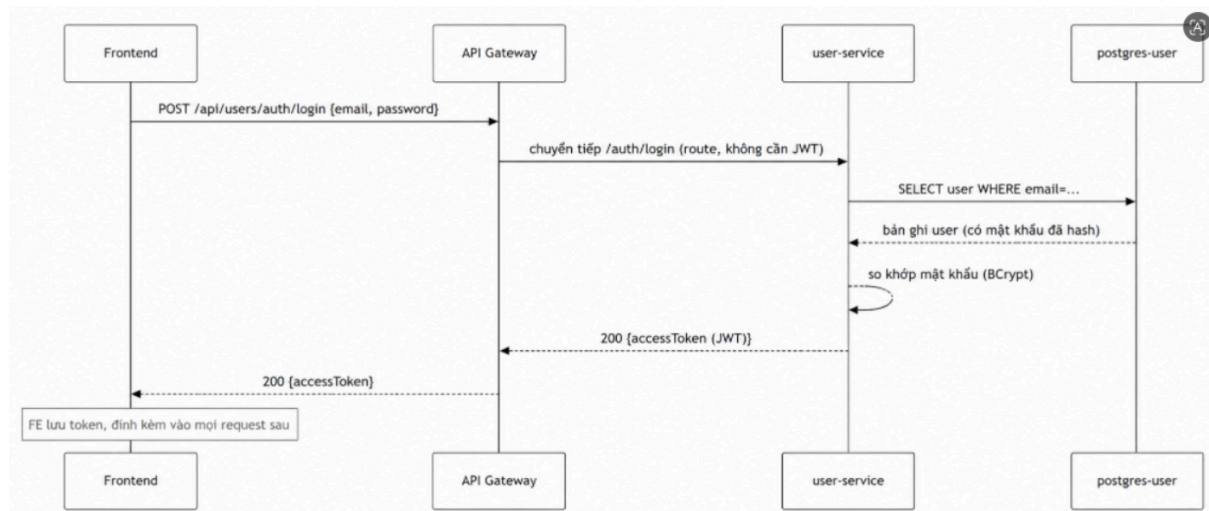
Thành phần	Loại	Tên	Mô tả
Exchange	Topic	order.events	Nhận event từ order-service, định tuyến theo routing key
Binding key	—	order.#	Khớp mọi routing key bắt đầu bằng chuỗi 'order.'
Queue	Durable	inventory.order-events	Hàng đợi bền vững dành riêng cho inventory-service
Queue	Durable	notification.order-events	Hàng đợi bền vững dành riêng cho notification-service
Exchange (DLX)	Direct	order.dlx	Dead Letter Exchange nhận message lỗi sau 3 lần retry thất bại

Queue (DLQ)	Durable	order-events.dlq	Lưu trữ message lỗi để admin kiểm tra và tái xử lý thủ công
-------------	---------	------------------	---

Mỗi consumer service có queue riêng, nên nhận được bản sao độc lập của cùng một sự kiện và xử lý hoàn toàn tách biệt. Hai consumer không cạnh tranh nhau trên cùng queue, đảm bảo cả hai đều nhận và xử lý đầy đủ mọi sự kiện.

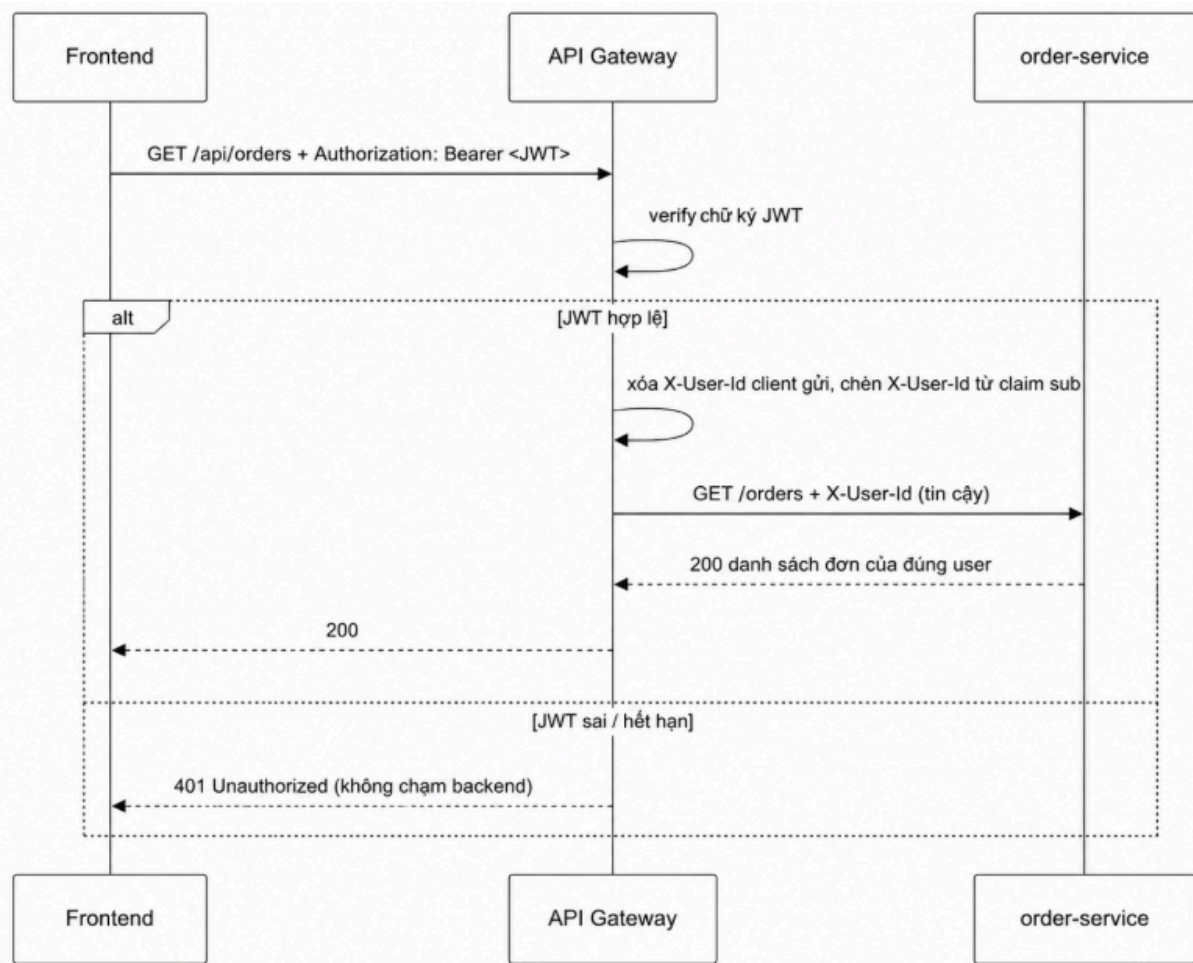
5.3. Luồng giao tiếp đồng bộ (REST)

5.3.1. Sequence Diagram luồng login



Hình 5.1. Sequence Diagram luồng login

Sau khi đăng nhập, mọi request cần quyền (Sequence Diagram luồng xem đơn hàng) đi như sau:

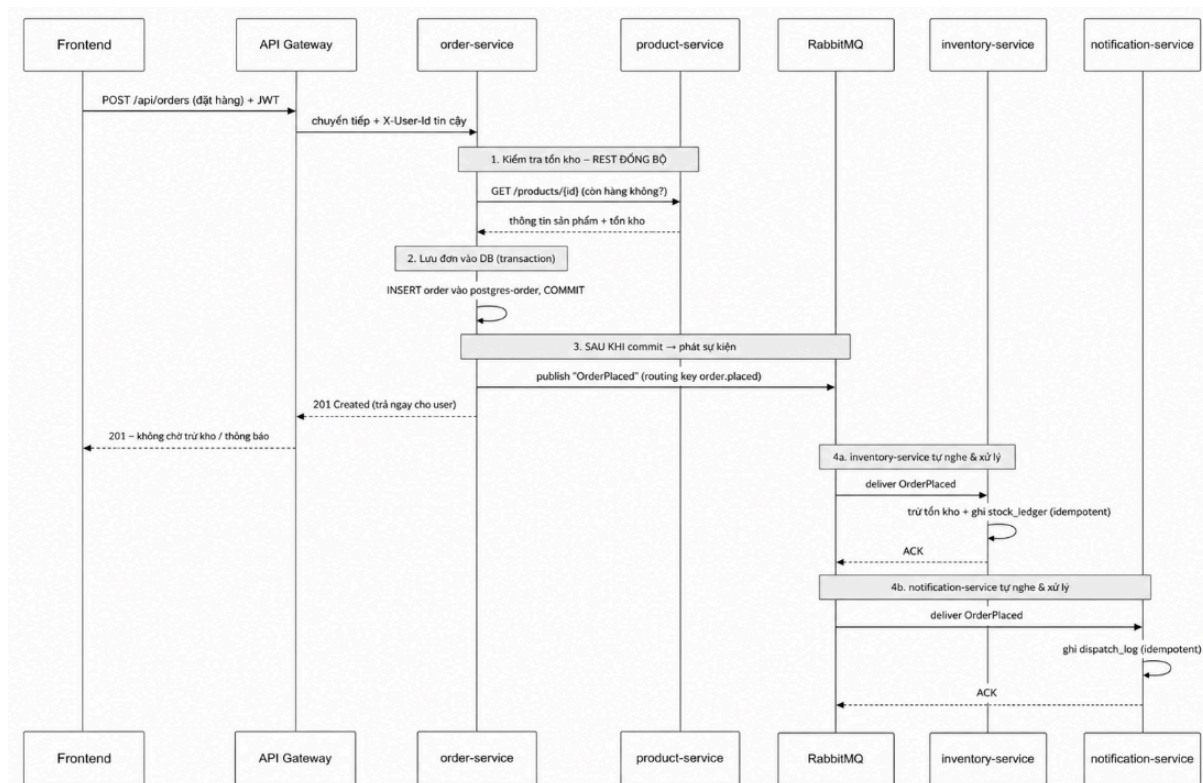


Hình 5.2. Sequence Diagram luồng xem đơn hàng

5.4. Luồng giao tiếp bất đồng bộ

5.4.1. Sequence Diagram luồng Đặt hàng

Đây là luồng quan trọng nhất:

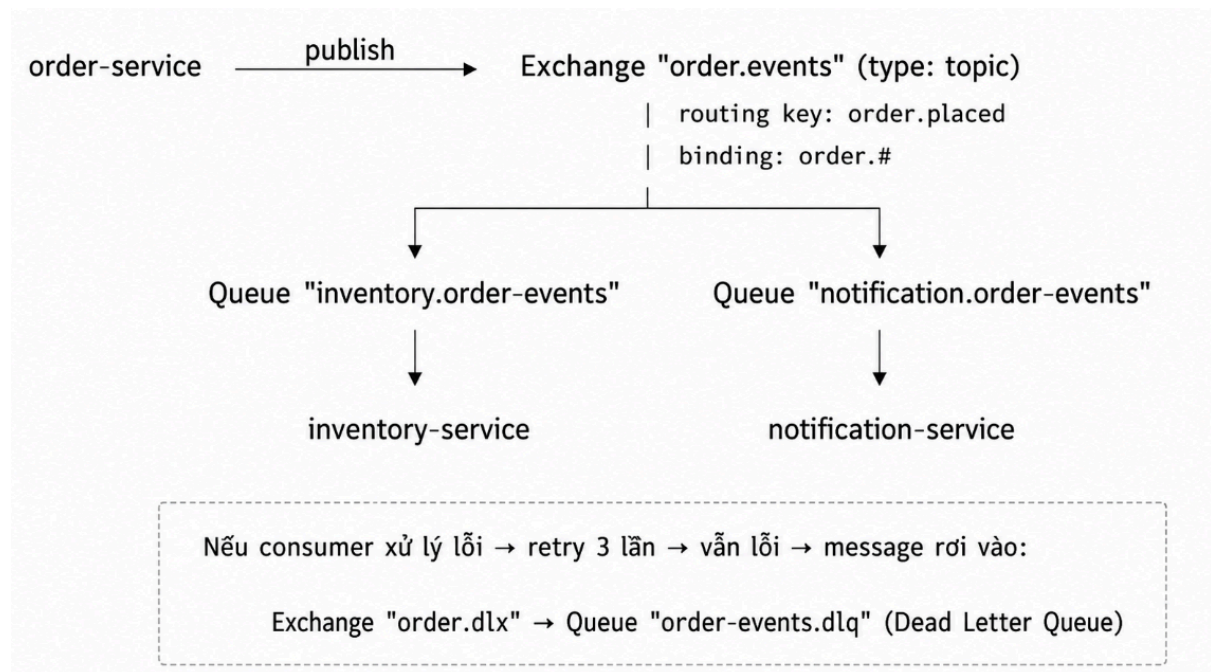


Hình 5.3. Sequence Diagram luồng giao tiếp bất đồng bộ

Vì sao tách đồng bộ / bất đồng bộ ở đây:

- Bước 1 (kiểm tra tồn kho) phải đồng bộ - nếu hết hàng, user cần biết ngay để không đặt được đơn lỗi.
- Bước 4 (trừ kho, ghi thông báo) bất đồng bộ - user không cần chờ. Đơn đã lưu thành công là đủ để phản hồi 201. Việc trừ kho và ghi log thông báo chạy nền. Nhờ vậy, nếu notification-service tạm thời không hoạt động, nhưng order-service vẫn tạo đơn bình thường. Message sẽ được giữ trong queue và được xử lý tiếp khi service hoạt động trở lại.

5.5. Topology RabbitMQ (chi tiết kỹ thuật)



5.6. Xử lý lỗi và Dead Letter Queue

Khi consumer gặp lỗi trong quá trình xử lý message (ví dụ: database tạm thời không khả dụng), RabbitMQ thực hiện retry tự động tối đa 3 lần. Nếu sau 3 lần vẫn thất bại, message được chuyển sang Dead Letter Exchange (order.dlx) và tích lũy trong queue order-events.dlq.

Cơ chế Dead Letter Queue đảm bảo: (1) message không bị mất vĩnh viễn khi consumer lỗi; (2) message lỗi không chặn các message bình thường phía sau trong queue; (3) admin có thể kiểm tra chi tiết message lỗi qua RabbitMQ Management UI tại localhost:15672 và tái xử lý thủ công sau khi khắc phục nguyên nhân lỗi.

Ngoài ra, mỗi consumer implement cơ chế idempotency bằng bảng processed_events: trước khi xử lý bất kỳ message nào, service kiểm tra eventId đã được xử lý thành công chưa. Nếu đã xử lý thì bỏ qua và gửi ACK ngay. Điều này đảm bảo dù message được RabbitMQ giao lại nhiều lần (do timeout ACK hoặc network lỗi), hệ thống vẫn chỉ thực hiện nghiệp vụ đúng một lần duy nhất.

CHƯƠNG 6. THỬ NGHIỆM VÀ ĐÁNH GIÁ

6.1. Môi trường thử nghiệm

Bảng 6.1. Môi trường thử nghiệm

Thành phần	Thông số cấu hình
Máy thử nghiệm	Windows 11, Intel Core i7, RAM 16GB, SSD 512GB
Docker Desktop	v4.x với WSL2 backend
Số container Docker	15 container (7 PostgreSQL, 1 RabbitMQ, 6 backend service, 1 frontend)
Trình duyệt kiểm thử	Google Chrome v124
Công cụ test API	Postman v11, PowerShell (curl/Invoke-RestMethod)
Framework E2E	Playwright (cấu hình sẵn trong thư mục sources/frontend/e2e)
Công cụ quan sát	RabbitMQ Management UI (localhost:15672, guest/guest)

6.2. Kịch bản kiểm thử

6.2.1. Kiểm thử luồng người dùng đầu cuối

Bảng 6.2. Kịch bản kiểm thử luồng người dùng

Kịch bản	Bước thực hiện	Kết quả mong đợi
Đăng ký và đăng nhập	POST /api/users/auth/register → POST /api/users/auth/login	Nhận JWT hợp lệ, lưu token vào cookie
Duyệt và tìm kiếm sản phẩm	GET /api/products?keyword=laptop&category=1&page=0	Danh sách sản phẩm lọc đúng, phân trang hoạt động

Thêm vào giỏ hàng	POST /api/orders/cart/items với productId và quantity	Giỏ hàng cập nhật, lưu bền vững trong DB
Áp dụng mã giảm giá	POST /api/orders/coupons/preview với couponCode	Hiển thị giá gốc và giá sau khi giảm
Đặt hàng thành công	POST /api/orders từ giỏ hàng hiện tại	Nhận 201 Created, đơn hàng xuất hiện trong lịch sử
Tồn kho giảm sau đặt hàng	Kiểm tra DB inventory và RabbitMQ Management UI	Tồn kho giảm đúng số lượng sau khoảng 1 giây
Ghi log thông báo	Kiểm tra bảng dispatch_log trong postgres-notification	Bản ghi thông báo đơn hàng mới được tạo thành công
Viết đánh giá sản phẩm	POST /api/products/{id}/reviews với rating và comment	Đánh giá lưu thành công, avg_rating cập nhật

6.2.2. Kiểm thử tính chịu lỗi của hệ thống phân tán

Bảng 6.3. Kịch bản kiểm thử tính chịu lỗi

Kịch bản	Cách thực hiện	Kết quả quan sát
Dừng notification-service khi đặt hàng	docker stop notification-service → đặt hàng mới	Đặt hàng vẫn thành công 201; message tích lũy trong queue
Khởi động lại notification-service	docker start notification-service	Message trong queue được xử lý; dispatch_log có bản ghi mới

Thử giả mạo X-User-Id	Gửi request với header X-User-Id: 9999 tự thêm vào	Gateway xóa header giả, chèn đúng userId từ JWT
Gửi JWT hết hạn	Gửi request với token đã hết hạn	Gateway trả 401; backend không nhận được request
Đặt hàng vượt tồn kho	Đặt số lượng lớn hơn tồn kho hiện có	Order-service trả 409 Conflict ngay lập tức
Gửi message trùng lặp (idempotency)	Publish cùng một OrderPlaced event hai lần lên queue	Inventory chỉ trừ kho một lần; event thứ hai bị bỏ qua
Truy cập endpoint admin với role User	Gọi GET /api/orders/admin/orders với token user thường	Service trả 403 Forbidden

6.3. Kết quả kiểm thử

Kết quả kiểm thử cho thấy hệ thống hoạt động đúng theo thiết kế trên tất cả kịch bản đề ra:

- 8/8 kịch bản kiểm thử luồng người dùng đều pass, dữ liệu nhất quán giữa các service và database.
- 7/7 kịch bản kiểm thử tính chịu lỗi đều cho kết quả đúng mong đợi.
- Khi notification-service bị dừng: message tích lũy an toàn trong queue RabbitMQ; khi service sống lại, toàn bộ message được xử lý đúng thứ tự, không có message nào bị mất.
- Cơ chế idempotency hoạt động chính xác: gửi lại cùng một OrderPlaced event hai lần, inventory chỉ bị trừ một lần duy nhất.
- Edge authentication tại Gateway ngăn hoàn toàn các request giả mạo header X-User-Id.

- Kiểm thử E2E với Playwright bao gồm 11 spec file (auth, catalog, cart, checkout, orders, profile, reviews, admin, chatbot, journey, message-queue) đều pass trên môi trường local.
- Thời gian khởi động toàn bộ hệ thống (docker-compose up từ image đã build) khoảng 30–45 giây; Flyway tự động migrate schema và seed dữ liệu mẫu thành công.

6.4. Đánh giá hệ thống

6.4.1. Ưu điểm đạt được

- Tách biệt nghiệp vụ rõ ràng: mỗi service có database riêng, không truy vấn chéo, coupling giữa các service ở mức tối thiểu.
- Khả năng mở rộng độc lập: có thể scale riêng order-service khi lượng đặt hàng tăng cao mà không cần scale các service khác.
- Tính chịu lỗi cao: một service ngừng hoạt động không kéo sập toàn bộ hệ thống; message queue là bộ đệm đáng tin cậy đảm bảo không mất dữ liệu.
- Bảo mật chặt chẽ: JWT verify tập trung tại Gateway, chống giả mạo định danh bằng cơ chế overwrite X-User-Id, phân quyền theo role ở cấp service.
- Khởi chạy đơn giản và nhất quán: docker-compose up khởi động toàn bộ 15 container; Flyway tự động tạo schema và seed dữ liệu mẫu.
- Khả năng quan sát: trace ID xuyên suốt request; RabbitMQ Management UI cung cấp giao diện trực quan theo dõi queue, exchange và message theo thời gian thực.

6.4.2. Hạn chế và hướng phát triển

- Chưa có distributed tracing đầy đủ (Zipkin/Jaeger): hiện chỉ có trace ID đơn giản trong header, chưa có giao diện trực quan theo dõi toàn bộ luồng request qua nhiều service.
- Chưa cấu hình circuit breaker: nếu product-service chậm phản hồi kéo dài, order-service có thể bị block theo. Cần tích hợp Resilience4j với fallback strategy.

- Thanh toán mock: chưa tích hợp cổng thanh toán thực (MoMo). Cấu trúc dữ liệu PaymentSession và PaymentTransaction đã sẵn sàng cho việc tích hợp.
- Chưa triển khai trên cloud: hệ thống chỉ chạy môi trường local. Hướng phát triển là deploy lên Kubernetes với Helm charts, auto-scaling và service mesh (Istio).
- Tìm kiếm sản phẩm dùng JPQL LIKE: chưa tích hợp Elasticsearch cho full-text search có độ chính xác cao hơn và hỗ trợ tìm kiếm mờ.

CHƯƠNG 7: HƯỚNG DẪN CÀI ĐẶT

Để hệ thống có thể vận hành đầy đủ, cần chuẩn bị môi trường phù hợp và thực hiện đúng quy trình khởi động các thành phần. Do dự án được xây dựng theo kiến trúc microservices, quá trình triển khai bao gồm nhiều service backend, cơ sở dữ liệu và message broker. Tuy nhiên, việc sử dụng Docker Compose giúp đơn giản hóa đáng kể quá trình cài đặt và vận hành.

7.1. Yêu cầu môi trường

Trước khi cài đặt hệ thống, cần đảm bảo máy tính đáp ứng các yêu cầu tối thiểu được trình bày trong Bảng

Bảng 7.1. Yêu cầu môi trường triển khai

Thành phần hệ thống	Yêu cầu cấu hình tối thiểu	Ghi chú / Mục đích sử dụng
Hệ điều hành	Windows 10/11, Linux hoặc macOS	Hỗ trợ môi trường ảo hóa ổn định.
Bộ xử lý (CPU)	Intel Core i5 hoặc tương đương	Đáp ứng năng lực tính toán phân tán.
Bộ nhớ RAM	Tối thiểu 8 GB (Khuyến nghị: 16 GB)	Đảm bảo không bị tràn bộ nhớ khi chạy nhiều container.
Dung lượng ổ cứng	Khả dụng tối thiểu 10 GB trống	Lưu trữ các Docker Image và dữ liệu của 7 PostgreSQL database.

Docker Desktop	Phiên bản hỗ trợ Docker Compose	Công cụ cốt lõi để đóng gói và khởi chạy toàn bộ hệ thống.
Node.js	Phiên bản 18 trở lên	Sử dụng khi cần khởi chạy hoặc debug riêng tầng Frontend Next.js.
Trình duyệt web	Google Chrome / Microsoft Edge / Firefox	Dùng để kiểm thử giao diện người dùng và truy cập RabbitMQ UI.

Hệ thống sử dụng nhiều container đồng thời, do đó cần đảm bảo Docker được cấp đủ tài nguyên để tránh tình trạng thiếu bộ nhớ trong quá trình vận hành.

7.2. Cài đặt và khởi động hệ thống

Bước 1. Tải mã nguồn dự án

Người dùng tiến hành sao chép mã nguồn từ kho lưu trữ GitHub về máy tính.

```
git clone https://github.com/dminhduc250136/ptit_htpt.git
cd ptit_htpt
```

Bước 2. Khởi động hệ thống bằng Docker Compose

Sau khi tải mã nguồn, sử dụng Docker Compose để khởi động toàn bộ hệ thống.

```
docker-compose up --build
```

Lệnh này sẽ thực hiện:

- build các image cần thiết
- khởi động frontend
- khởi động API Gateway
- khởi động các backend microservice
- khởi động PostgreSQL cho từng service
- khởi động RabbitMQ

Quá trình khởi động có thể mất vài phút tùy theo cấu hình máy.

Bước 3. Kiểm tra trạng thái container

Sau khi hệ thống khởi động, kiểm tra danh sách container đang chạy:

```
docker ps
```

Nếu cài đặt thành công, hệ thống sẽ hiển thị đầy đủ các container tương ứng với frontend, API Gateway, backend services, RabbitMQ và PostgreSQL.

7.3. Truy cập hệ thống

Sau khi hệ thống vận hành thành công, người dùng có thể truy cập các thành phần theo Bảng.

Bảng 7.3. Địa chỉ truy cập hệ thống

Thành phần hệ thống	Địa chỉ URL truy cập
Frontend (Next.js)	http://localhost:3000 (Giao diện Người dùng và Admin)
API Gateway	http://localhost:8080 (Cổng vào duy nhất của REST API)
Swagger UI	http://localhost:8080/swagger-ui.html (Tài liệu đặc tả Endpoint)
RabbitMQ Management UI	http://localhost:15672 (Tài khoản mặc định: <i>guest / guest</i>)

RabbitMQ Management UI cho phép theo dõi trạng thái queue, exchange và message trong quá trình vận hành.

7.4. Hướng dẫn sử dụng cơ bản

- Đối với người dùng thông thường

Quy trình sử dụng hệ thống gồm các bước cơ bản:

Bước 1: Truy cập giao diện hệ thống tại frontend.

Bước 2: Đăng ký tài khoản mới hoặc đăng nhập bằng tài khoản đã có.

Bước 3: Duyệt danh sách sản phẩm, tìm kiếm hoặc lọc theo nhu cầu.

Bước 4: Thêm sản phẩm vào giỏ hàng.

Bước 5: Áp dụng mã giảm giá nếu có.

Bước 6: Tiến hành đặt hàng và theo dõi lịch sử đơn hàng.

Bước 7: Đánh giá sản phẩm sau khi mua hàng.

Đối với quản trị viên

Quản trị viên có thể:

- quản lý danh mục sản phẩm
- thêm, sửa, xóa sản phẩm
- quản lý người dùng
- theo dõi đơn hàng
- cập nhật trạng thái đơn hàng
- quản lý mã giảm giá
- giám sát hoạt động hệ thống

7.5. Dừng hệ thống

- Để dừng toàn bộ hệ thống: `docker-compose down`
- Nếu muốn xóa toàn bộ dữ liệu volume: `docker-compose down -v`

Lưu ý rằng thao tác này sẽ xóa dữ liệu đã lưu trong cơ sở dữ liệu container.

7.6. Một số lỗi thường gặp

Docker không đủ bộ nhớ

Biểu hiện: container khởi động thất bại hoặc tự dừng.

Cách khắc phục: tăng RAM cấp cho Docker Desktop.

Port đã được sử dụng

Biểu hiện: lỗi không bind được cổng.

Cách khắc phục: dừng ứng dụng đang sử dụng các cổng:

- 3000
- 8080
- 15672

Service chưa khởi động hoàn toàn

Biểu hiện: frontend không kết nối được backend.

Cách khắc phục: chờ thêm vài phút hoặc kiểm tra log:

`docker logs <container_name>`

KẾT LUẬN

Đồ án đã xây dựng thành công một sàn thương mại điện tử hoàn chỉnh theo kiến trúc Microservices, minh họa đầy đủ các nguyên lý cốt lõi của hệ thống phân tán. Hệ thống bao gồm 6 backend microservice độc lập (user, product, order, payment, inventory, notification), 1 API Gateway, 1 frontend Next.js, 7 database PostgreSQL riêng biệt và 1 RabbitMQ broker, được đóng gói hoàn toàn bằng Docker Compose.

Các kết quả nổi bật đã đạt được: triển khai đầy đủ mô hình database-per-service đảm bảo tính độc lập dữ liệu; cơ chế xác thực JWT tập trung tại Gateway với khả năng chống giả mạo định danh hiệu quả; luồng giao tiếp bất đồng bộ qua RabbitMQ với idempotency và Dead Letter Queue đảm bảo không mất message; và hệ thống chứng minh tính chịu lỗi độc lập khi một service ngừng hoạt động trong khi toàn bộ hệ thống vẫn tiếp tục phục vụ.

Thông qua quá trình thực hiện đồ án, nhóm đã củng cố hiểu biết sâu sắc về thiết kế microservices, giao tiếp service-to-service (đồng bộ và bất đồng bộ), message-driven architecture, bảo mật tập trung tại biên và tổ chức dữ liệu trong hệ thống phân tán. Hệ thống không chỉ đáp ứng đầy đủ yêu cầu đề ra mà còn là nền tảng vững chắc để tiếp tục phát triển theo hướng cloud-native với Kubernetes, service mesh và observability platform hoàn chỉnh.

TÀI LIỆU THAM KHẢO

- [1] S. Newman, *Building Microservices: Designing Fine-Grained Systems*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2021.
- [2] C. Richardson, *Microservices Patterns: With Examples in Java*. Shelter Island, NY, USA: Manning Publications, 2018.
- [3] “Spring Cloud Gateway Documentation,” Spring. [Online]. Available: [Spring Cloud Gateway Documentation](#). [Accessed: May 2026].
- [4] “RabbitMQ Documentation — Tutorials and Best Practices,” RabbitMQ. [Online]. Available: [RabbitMQ Documentation](#). [Accessed: May 2026].
- [5] “Next.js 14 Documentation,” Next.js. [Online]. Available: [Next.js 14 Documentation](#). [Accessed: May 2026].
- [6] “Flyway Database Migration Documentation,” Flyway. [Online]. Available: [Flyway Documentation](#). [Accessed: May 2026].
- [7] “Docker Compose Reference Documentation,” Docker. [Online]. Available: [Docker Compose Documentation](#). [Accessed: May 2026].
- [8] M. Fowler, *Patterns of Enterprise Application Architecture*. Boston, MA, USA: Addison-Wesley, 2002.
- [9] “RFC 7519: JSON Web Token (JWT),” Internet Engineering Task Force (IETF), May 2015. [Online]. Available: [RFC 7519 JWT](#). [Accessed: May 2026].